

Gabriel Piangers

11/06/2026

Lab 8 - Uso de LLMs

Exercício 1 – Escolha da ferramenta

Ferramenta utilizada: Gemini 3.5 Flash, versão gratuita

Data de uso: 11/06/2026

Possui acesso a internet?: Sim

Exercício 2 – Primeira tarefa: explicação conceitual

Prompt simples:

Explique brevemente o calculo de Cross Entropy Loss em redes neurais

Resposta:

— INICIO DA RESPOTA —

A **Cross-Entropy Loss** (ou Entropia Cruzada) é a função de perda mais utilizada em problemas de **classificação**, especialmente quando trabalhamos com redes neurais.

O principal objetivo dela é medir o quão “distante” a previsão da rede neural está do valor real esperado. Quanto pior a previsão da rede, maior será o valor da perda.

A Ideia por Trás do Cálculo

Imagine um cenário de classificação onde o resultado real (o *target*) é representado no formato **One-Hot Encoding** (um vetor de zeros com o valor 1 na posição correta). A rede neural, por sua vez, passa suas saídas por uma função (geralmente a **Softmax**) para gerar uma distribuição de **probabilidades** para cada classe.

A fórmula geral da Cross-Entropy para uma única amostra é:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Onde:

- C é o número total de classes.
- y_i é o valor real da classe i (geralmente 0 ou 1).
- $\hat{y}_i$ é a probabilidade predita pela rede para a classe i (um valor entre 0 e 1).

Por que ela é tão eficiente?

Como os valores reais (y_i) são 0 para todas as classes incorretas e 1 para a classe correta, a fórmula simplifica drasticamente na prática. Ela foca **apenas na probabilidade que a rede previu para a classe certa**.

Se a classe correta for a classe k , o cálculo se resume a:

$$L = -\log(\hat{y}_k)$$

- **Se a rede estiver muito confiante e certa:** A probabilidade $\hat{y}_k$ será próxima de 1. Como $\log(1) = 0$, a perda L será praticamente **0**.
- **Se a rede estiver errada:** A probabilidade $\hat{y}_k$ será próxima de 0. Como o logaritmo de um número muito próximo de zero tende ao infinito negativo ($-\infty$), o sinal negativo na fórmula transforma isso em uma **perda gigantesca (penalidade alta)**.

Em resumo

A Cross-Entropy pune severamente os erros cometidos com “certeza” (quando a rede bota muita probabilidade na classe errada) e recompensa a rede quando ela direciona a maior probabilidade possível para a classe correta, forçando a otimização dos pesos durante o *backpropagation*.

— FIM DA RESPOTA —

Prompt elaborado:

Você é um especialista em Redes Neurais e em Deep learning.

Estou começando a aprender sobre redes neurais durante o curso de Ciência da Computação e preciso que você explique o funcionamento da função Cross Entropy Loss neste contexto. Mais especificamente preciso que detalhe as seguintes informações sobre o assunto:

- Fórmula matemática aplicada na função, seu funcionamento e propriedades.

- Para quais situações o uso da função é recomendado e contraindicado.
- Comparação entre a função de Cross Entropy Loss e de Mean Squared Error

Resposta:

— INICIO DA RESPOTA —

Com certeza! Vamos elevar o nível do detalhamento. Como você está cursando Ciência da Computação, vamos analisar a **Cross-Entropy Loss (CE)** não apenas de forma intuitiva, mas abordando seu comportamento matemático, propriedades estatísticas e comparando-a diretamente com a **Mean Squared Error (MSE)** sob a ótica de otimização por gradiente.

1. Fórmula Matemática, Funcionamento e Propriedades

A Cross-Entropy tem suas raízes na Teoria da Informação de Shannon. Ela mede a “distância” entre duas distribuições de probabilidade: a distribuição real P (os rótulos corretos) e a distribuição predita Q (as saídas da sua rede neural).

A Fórmula Geral (Multiclasse)

Para um problema com C classes, a perda para uma única amostra é dada por:

$$L_{CE} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Onde:

- $y_i \in \{0, 1\}$ é o indicador binário (via *One-Hot Encoding*) se a classe i é o rótulo correto.
- $\hat{y}_i \in (0, 1)$ é a probabilidade estimada pela rede para a classe i , tipicamente obtida após a camada de ativação **Softmax**:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

(Sendo z o vetor de logits/saídas brutas da última camada linear).

Funcionamento Prático

Como $y_i = 0$ para todas as classes incorretas e $y_i = 1$ para a classe correta (vamos chamá-la de classe k), a somatória colapsa para:

$$L_{CE} = -\log(\hat{y}_k)$$

Propriedades Matemáticas Cruciais

1. **Assimetria:** $H(P, Q) \neq H(Q, P)$. Nós medimos a divergência da predição Q em relação ao alvo fixo P .
 2. **Não-negatividade:** Como $\hat{y}_k \in (0, 1]$, o valor de $\log(\hat{y}_k)$ é sempre menor ou igual a zero. O sinal negativo garante que $L_{CE} \geq 0$.
 3. **Penalização Assintótica:** Quando $\hat{y}_k \rightarrow 0$ (o modelo erra com convicção), $L_{CE} \rightarrow \infty$. Isso cria uma força de gradiente massiva para corrigir o modelo. Se $\hat{y}_k \rightarrow 1$, $L_{CE} \rightarrow 0$.
 4. **Convexidade:** Quando combinada com a ativação Softmax (ou Sigmoid no caso binário), a função de perda em relação aos *logits* (z) se torna convexa, garantindo que o otimizador encontre um mínimo global facilmente sem ficar preso em platôs locais causados pela perda em si.
-

2. Recomendações e Contraindicações

Quando é Recomendada?

- **Classificação Multiclasse e Binária:** É o padrão ouro para classificação (ex: classificar imagens no ImageNet, detecção de spam, análise de sentimento).
- **Modelagem Probabilística (Máxima Verossimilhança):** Minimizar a Cross-Entropy é matematicamente equivalente a maximizar a log-verossimilhança (*Maximum Likelihood Estimation* - *MLE*) dos dados sob um modelo multinomial.
- **Problemas de Aprendizado por Reforço:** Na otimização de políticas (Policy Gradient), para maximizar a probabilidade de ações que geram recompensas.

Quando é Contraindicada?

- **Problemas de Regressão:** Se o seu objetivo é prever um valor contínuo (ex: preço de uma casa, temperatura), a CE não pode ser usada, pois não há uma distribuição de probabilidade categórica discreta na saída.
 - **Dados Altamente Ruidosos (Labels Incorretos):** Como a CE penaliza severamente erros convictos ($\log(\hat{y}) \rightarrow \infty$), se o seu dataset tiver rótulos errados de forma sistemática, o modelo vai distorcer drasticamente seus pesos para tentar acertar o ruído.
 - **Gradientes Explosivos em Saídas Extremas:** Se não houver uma estabilização numérica no código (como aplicar `LogSoftmax`), o cálculo direto de $\log(0)$ estoura para NaN ou causa instabilidade numérica.
-

3. Cross-Entropy Loss vs. Mean Squared Error (MSE)

Embora o MSE (Erro Quadrático Médio) possa, teoricamente, ser usado para classificação tratando as probabilidades como valores contínuos, ele é amplamente rejeitado para essa tarefa em Deep Learning. Vamos entender o porquê:

Tabela Comparativa

Critério	Cross-Entropy Loss	Mean Squared Error (MSE)
Fórmula (Amostra)	$L = -\log(\hat{y}_k)$	$L = \frac{1}{C} \sum_{i=1}^C (y_i - \hat{y}_i)^2$
Domínio Principal	Classificação (Categórico)	Regressão (Contínuo)
Ativação Final	Softmax (Multiclasse) / Sigmoid (Binário)	Linear / Identidade
Dinâmica do Gradiente	Linear em relação ao erro (evita saturação)	Decai quando o modelo está “muito errado” (saturação)

O Fenômeno do “Gradiente Sumindo” (Saturação)

A principal razão acadêmica para **não** usar MSE em classificação reside na derivada da perda em relação aos pesos da rede.

Imagine um problema de classificação binária usando a função de ativação **Sigmoid** ($\sigma(z)$). A derivada da Sigmoid possui uma propriedade: ela é próxima de zero quando o valor de z é muito grande ou muito pequeno ($\sigma'(z) \rightarrow 0$).

1. **Se usarmos MSE:** A derivada da perda MSE em relação ao logit z carrega o termo da derivada da própria Sigmoid:

$$\frac{\partial L_{MSE}}{\partial z} = (\hat{y} - y) \cdot \sigma'(z)$$

Se a rede fizer uma predição **muito errada** (ex: $y = 1$ mas a rede previu $\hat{y} = 0$), o termo z será um número negativo muito grande. Nesse ponto, $\sigma'(z)$ é praticamente **zero**. O gradiente resulta em zero, a rede “trava” e não consegue aprender com seu pior erro. 2. **Se usarmos Cross-Entropy:** Ao calcular a derivada da CE em relação ao logit z , acontece um cancelamento matemático elegante. A derivada da Sigmoid/Softmax no denominador se cancela com o termo da perda, resultando simplesmente em:

$$\frac{\partial L_{CE}}{\partial z} = \hat{y} - y$$

Note que o gradiente é **diretamente proporcional ao tamanho do erro**. Se a rede errar feio ($\hat{y} = 0$ e $y = 1$), o gradiente será máximo (-1), forçando uma atualização de pesos agressiva e imediata. Não há saturação de gradiente na camada de saída.

— FIM DA RESPOTA —

Comparação das respostas

Apesar do primeiro prompt ser simples e pouco detalhado a LLM ainda assim conseguiu identificar os principais tópicos a serem explicados pelo assunto e soube resumir bem o conteúdo. Entretanto, através do prompt mais elaborado conseguimos especificar melhor o que queremos receber, o que gera uma resposta mais adequada e específica a nossa necessidade.

Exercício 3 – Segunda tarefa: geração de código

Prompt:

Faça um programa que leia um arquivo csv

Resposta:

— INICIO DA RESPOTA —

Criar um leitor de arquivo CSV em Python é uma tarefa simples e direta, graças à biblioteca nativa `csv`.

Abaixo, preparei um exemplo prático. Ele abre o arquivo com segurança usando o gerenciador de contexto `with` (que garante que o arquivo seja fechado mesmo se ocorrer um erro) e imprime o conteúdo linha por linha.

Código em Python

```
import csv

# Substitua pelo caminho e nome do seu arquivo .csv
caminho_arquivo = "dados.csv"

try:
    # O parâmetro encoding='utf-8' garante a leitura correta de acentos eç
    with open(caminho_arquivo, mode="r", encoding="utf-8") as arquivo:
        # Cria o leitor de CSV
        leitor_csv = csv.reader(arquivo)

        # Se o seu arquivo tiver um cabeçalho, você pode descomentar a linha abaixo para pu
```

```

        # cabecalho = next(leitor_csv)

        # Itera sobre cada linha do arquivo
        for linha in leitor_csv:
            # 'linha' é uma lista onde cada elemento é uma coluna
            print(linha)

except FileNotFoundError:
    print(f"Erro: O arquivo '{caminho_arquivo}' não foi encontrado.")
except Exception as e:
    print(f"Ocorreu um erro inesperado: {e}")

```

Alternativa Avançada: Usando a biblioteca Pandas

Se o seu objetivo for analisar ou manipular os dados após a leitura, a biblioteca **Pandas** é muito mais poderosa e recomendada.

Nota: Para usar este método, você precisa instalar a biblioteca antes rodando `pip install pandas` no seu terminal.

```

import pandas as pd

caminho_arquivo = "dados.csv"

try:
    # Lê o arquivo e já o transforma em um DataFrame (tabela)
    df = pd.read_csv(caminho_arquivo)

    # Exibe as primeiras 5 linhas do arquivo
    print(df.head())

except FileNotFoundError:
    print(f"Erro: O arquivo '{caminho_arquivo}' não foi encontrado.")

```

Qual é a estrutura do seu arquivo CSV e o que você pretende fazer com esses dados depois de lê-los?

— FIM DA RESPOTA —

Explicação do código

O assistente gerou 2 códigos distintos, ambos que leem um arquivo .csv e imprimem seu conteúdo na tela. O primeiro código utiliza a biblioteca csv que lê o arquivo através da função `reader()` e imprime na tela linha por linha. O segundo código utiliza a biblioteca Pandas, que tecnicamente faz a mesma

coisa através da função `read_csv()`, porém retorna um `DataFrame`, um tipo de dado nativo da biblioteca `Pandas` que representa uma tabela. Nesse caso `print(df.head())` imprime as 5 primeiras linhas da tabela no terminal.

Testes

Código 1, usando a biblioteca `csv`

```
['id', 'nome', 'email', 'cidade', 'idade']
['1', 'Ana Silva', 'ana.silva@email.com', 'São Paulo', '28']
['2', 'Carlos Souza', 'carlos.souza@email.com', 'Rio de Janeiro', '35']
['3', 'Mariana Costa', 'mariana.costa@email.com', 'Belo Horizonte', '22']
['4', 'Pedro Alves', 'pedro.alves@email.com', 'Curitiba', '41']
['5', 'Juliana Lima', 'juliana.lima@email.com', 'Porto Alegre', '30']
```

Código 2, usando `Pandas`

	id	nome	email	cidade	idade
0	1	Ana Silva	ana.silva@email.com	São Paulo	28
1	2	Carlos Souza	carlos.souza@email.com	Rio de Janeiro	35
2	3	Mariana Costa	mariana.costa@email.com	Belo Horizonte	22
3	4	Pedro Alves	pedro.alves@email.com	Curitiba	41
4	5	Juliana Lima	juliana.lima@email.com	Porto Alegre	30

Análise crítica da solução

Ambas as soluções funcionam corretamente e possuem verificações em casos de erro. Oferecer 2 soluções possíveis é uma boa resposta em casos onde fatores importantes não são especificados. Entretanto, ambas as respostas são claramente genéricas e possuem pouca explicação de como o código funciona ou por que foi feito desta maneira e não de outra.

Exercício 4 – Melhorando o prompt

Prompt

Faça um programa em C que leia um arquivo `csv` chamado “clientes.csv” e imprima o resultado na tela em um formato de tabela.

Crie uma struct para representar as informações de cada linha no arquivo e ao fazer a leitura armazene tudo em um único vetor.

Os campos presentes no `csv` são: `id`, `nome`, `email`, `cidade` e `idade`.

A quantidade de entradas no arquivo é desconhecida.

Se possível separe o código em funções simples e intuitivas que facilitem o reuso da lógica aplicada.

Resposta

— INICIO DA RESPOTA —

Aqui está uma solução em C estruturada exatamente como você pediu.

Como a quantidade de entradas no arquivo é desconhecida, a melhor abordagem em C para gerenciar o vetor é a **alocação dinâmica de memória** (`realloc`). Dessa forma, o programa aumenta o tamanho do vetor conforme lê novas linhas, evitando desperdício de memória ou estouro de limite.

Para deixar o código limpo, dividi a lógica em funções dedicadas para abrir o arquivo, ler/parsear os dados e imprimir a tabela.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LINHA 256
#define CAPACIDADE_INICIAL 10

// Struct que representa cada linha do CSV
typedef struct {
    int id;
    char nome[50];
    char email[80];
    char cidade[50];
    int idade;
} Cliente;

// Protótipos das funções
Cliente* ler_clientes_csv(const char* nome_arquivo, int* total_clientes);
void exibir_tabela_clientes(const Cliente* clientes, int total);

int main() {
    int total_clientes = 0;

    // Chama a função de leitura
    Cliente* vetor_clientes = ler_clientes_csv("clientes.csv", &total_clientes);

    // Se o vetor não for nulo e houver clientes, exibe e libera a memória
    if (vetor_clientes != NULL) {
        if (total_clientes > 0) {
            exibir_tabela_clientes(vetor_clientes, total_clientes);
        } else {
            printf("O arquivo está vazio ou não contém dados válidos.\n");
        }
    }
}
```

```

    }

    // Regra de ouro da alocação dinâmica: liberando a memória alocada
    free(vetor_clientes);
}

return 0;
}

/**
 * Função responsável por abrir o CSV, processar linha por linha
 * e armazenar dinamicamente os dados em um vetor.
 */
Cliente* ler_clientes_csv(const char* nome_arquivo, int* total_clientes) {
    FILE* arquivo = fopen(nome_arquivo, "r");
    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo %s.\n", nome_arquivo);
        return NULL;
    }

    int capacidade = CAPACIDADE_INICIAL;
    int contador = 0;

    // Aloca a memória inicial para o vetor de structs
    Cliente* vetor = (Cliente*) malloc(capacidade * sizeof(Cliente));
    if (vetor == NULL) {
        printf("Erro de memória ao inicializar o vetor.\n");
        fclose(arquivo);
        return NULL;
    }

    char linha[MAX_LINHA];

    // Pula o cabeçalho do CSV (id, nome, email, cidade, idade)
    if (fgets(linha, sizeof(linha), arquivo) == NULL) {
        // Arquivo totalmente vazio
        fclose(arquivo);
        return vetor;
    }

    // Lê o arquivo linha por linha até o fim (EOF)
    while (fgets(linha, sizeof(linha), arquivo) != NULL) {

        // Se o vetor estiver cheio, dobra a capacidade dele na memória
        if (contador >= capacidade) {
            capacidade *= 2;

```

```

        Cliente* temp = (Cliente*) realloc(vetor, capacidade * sizeof(Cliente));
        if (temp == NULL) {
            printf("Erro de memória ao expandir o vetor.\n");
            free(vetor);
            fclose(arquivo);
            return NULL;
        }
        vetor = temp;
    }

    // Remove a quebra de linha (\n ou \r\n) que o fgets traz
    linha[strcspn(linha, "\r\n")] = '\0';

    // Ponteiro auxiliar para preencher a struct atual
    Cliente* c = &vetor[contador];

    /* * Usamos sscanf para extrair os dados formatados por vírgula.
     * O formato %49[^,] lê até 49 caracteres ou até encontrar uma vírgula.
     */
    int campos_lidos = sscanf(linha, "%d,%49[^,],%79[^,],%49[^,],%d",
                              &c->id, c->nome, c->email, c->cidade, &c->idade);

    // Se a linha foi lida com sucesso (todos os 5 campos preenchidos)
    if (campos_lidos == 5) {
        contador++;
    }
}

fclose(arquivo);
*total_clientes = contador; // Retorna a quantidade lida por referência
return vetor;
}

/**
 * Função responsável por formatar e exibir o vetor na tela como uma tabela organizada.
 */
void exibir_tabela_clientes(const Cliente* clientes, int total) {
    // Linha divisória formatada
    const char* divisoria = "+-----+-----+-----+-----+-----+";

    printf("\n%s\n", divisoria);
    printf("| %-2s | %-20s | %-30s | %-15s | %-4s |\n", "ID", "Nome", "E-mail", "Cidade", "Idade");
    printf("%s\n", divisoria);

    for (int i = 0; i < total; i++) {
        printf("| %-2d | %-20s | %-30s | %-15s | %-4d |\n",

```

```

        clientes[i].id,
        clientes[i].nome,
        clientes[i].email,
        clientes[i].cidade,
        clientes[i].idade);
    }

    printf("%s\n\n", divisoria);
    printf("Total de registros exibidos: %d\n", total);
}

```

Destaques do Código:

- **sscanf(..., "%d,%49[^\n],...")**: Essa é uma forma muito eficiente de ler arquivos .csv simples em C. O padrão `[^\n,]` diz para o programa ler “tudo que não for uma vírgula”, o que resolve perfeitamente a separação das strings. Os limites (como `%49[^\n,]`) evitam estouro de buffer (*buffer overflow*) caso o nome no arquivo seja maior do que o espaço da struct.
- **realloc Dinâmico**: Ele começa alocando espaço para 10 clientes. Se o arquivo tiver 11, o programa dobra para 20. Se tiver 21, dobra para 40, e assim por diante. Isso resolve o problema de a quantidade de entradas ser “desconhecida” sem fixar um número absurdamente grande no código.
- **Tabela Alinhada**: Na função `exibir_tabela_clientes`, o modificador `%-20s` garante que o texto seja alinhado à esquerda com um espaçamento fixo de 20 caracteres, mantendo as bordas da tabela retas, não importa o tamanho do nome ou email.

Como testar:

Certifique-se de criar um arquivo chamado `clientes.csv` na **mesma pasta** do executável com um conteúdo semelhante a este (sem espaços extras entre as vírgulas):

```

id,nome,email,cidade,idade
1,Ana Silva,ana.silva@email.com,Porto Alegre,28
2,Bruno Souza,bruno@email.com,Sao Paulo,34
3,Carlos Andrade,carlos.a@email.com,Curitiba,22

```

— FIM DA RESPOTA —

Comparação entre as duas respostas

No primeiro prompt haviam muitas informações não especificadas o que fez com que a resposta se tornasse genérica e pouco elaborada, entretanto, vale

ressaltar a capacidade de derivar informações implícitas e de tomada de decisão da ferramenta.

Primeiro, o assistente precisou escolher uma linguagem de programação para realizar a tarefa, neste caso ele verificou o problema e percebeu que se tratava de análise de dados (leitura de dados), o que é comumente realizado em Python. Por isso, optou por esta linguagem. Em segundo momento, foi preciso decidir como abordar o problema. Já que não havia nenhuma especificação sobre qual a finalidade da aplicação, a ferramenta elencou as 2 opções mais comuns e genéricas de se realizar essa tarefa, que seriam as versões com a biblioteca padrão do Python csv (mais prática, intuitiva e não requer instalações) e a versão usando a biblioteca Pandas (mais robusta e amplamente utilizado em análise de dados).

Estas decisões tomadas pelo assistente foram muito bem pensadas e executadas, exemplificando a qualidade da ferramenta. Entretanto, se o que queríamos era um código em C então essa resposta é completamente inútil para nós.

Já no segundo prompt, ao especificar exatamente o que queríamos, qual a linguagem, como é a estrutura do arquivo e como o programa deve ser feito e estruturado, obtivemos uma resposta muito mais coerente com as nossas necessidades e exatamente no formato que esperávamos, sem ser preciso fazer nenhuma (ou poucas) alterações.

Exercício 5 - Verificação crítica

prompt escolhido: Faça um programa que leia um arquivo csv (exercício 3)

1. A resposta estava correta?

Sim, o código escrito estava correto e funcionando. Porém, não era exatamente o que eu queria. Por falta de informações a ferramenta optou por uma abordagem genérica e comum, de maneira que tivesse mais chance de dar uma resposta útil, porém isso obviamente não é tão bom quanto uma resposta completa e voltada à minha necessidade.

2. Havia informações incompletas ou incorretas?

Informações como o nome do arquivo precisavam ser alteradas manualmente na resposta fornecida. Em geral a resposta não estava pronta, era apenas um exemplo de implementação, mas que necessariamente precisaria de adaptações para ser utilizada.

3. Como a resposta foi verificada?

Fiz um arquivo .csv genérico e alterei apenas a linha que definia o nome do arquivo, para que não houvesse nenhum erro, executei ambas as versões do programa, comparando os resultados. Conforme mencionado foi preciso realizar modificações, ainda que mínimas ao programa.

4. A LLM demonstrou excesso de confiança?

Não! Pelo contrário, ela teve consciência de que não tinha informações suficientes para realizar a tarefa da melhor maneira. Inclusive, terminando sua resposta com uma pergunta, justamente para esclarecer suas dúvidas.

5. Como o prompt poderia ser melhorado?

O prompt podia ser melhorado, conforme elaborado no exercício 4, especificando os resultados desejados e as restrições a serem seguidas e, opcionalmente, explicando o contexto em qual o programa seria utilizado.

Exercício 6 - Boas práticas de uso

1. Por que não devemos copiar respostas sem revisão?

As respostas em geral podem possuir erros (não foi o caso nos prompts aqui apresentados porém acontece com mais frequência do que se imagina), sem falar que muitas vezes é preciso fazer pequenos ajustes para que a resposta se adeque de fato ao seu caso de uso.

2. Por que é importante informar os prompts utilizados?

O prompt é o que guia as respostas das LLMs, ao alterar poucas palavras em seu conteúdo já conseguimos obter respostas completamente diferentes. Nesse sentido, o prompt é de grande importância para analisar a qualidade da resposta.

3. Em quais situações as LLMs ajudam no aprendizado?

As LLMs foram treinadas com muitos textos explicativos e descritivos, por isso são ótimas em explicar e detalhar processos, podendo inclusive gerar relatórios voltados para suas dificuldades e escrito de uma maneira pensada no seu contexto específico. Em geral, uma boa abordagem para se usar a ferramenta no aprendizado é tratar-la como um colega veterano na sua área, fazendo perguntas sempre que precisar esclarecer alguma informação.

4. Em quais situações elas podem atrapalhar?

Existem várias situações onde o uso de LLMs é contraindicado, por exemplo, em sistemas críticos, onde o erro pode gerar consequências, ou em sistemas que lidem com informações sensíveis. Além disso, seu uso pode ser ineficiente em áreas onde a ferramenta não foi bem treinada, geralmente por falta de dados.

5. O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Utilizar uma LLM como apoio é usá-la de maneira a lhe beneficiar e não de maneira a lhe substituir. Ou seja, a mente por trás de tudo ainda deve ser o

usuário e este deve compreender completamente a tarefa em questão, assim como o estado da implementação atual. Desta maneira, o usuário não pode se tornar dependente da ferramenta e precisa ter consciência do que está sendo feito em cada etapa.

Reflexão Final

1. O que você aprendeu sobre interação com LLMs?

LLMs são ferramentas muito uteis em certas tarefas, porém suas respostas podem acabar sendo inconsistentes, sendo sempre necessário a validação. Práticas que podem mitigar isso é a elaboração de um prompt detalhado e objetivo e a inserção de contexto para guiar as respostas.

2. Como a qualidade do prompt influenciou os resultados?

Prompts mais detalhados levam a respostas mais precisas e personalizadas às nossas necessidades, enquanto prompts amplos costumam ser menos produtivos e mais genéricos.

3. Em quais situações a ferramenta foi mais útil?

A ferramenta foi muito bem na escrita de código, mas também se saiu bem ao detalhar informações sobre um determinado assunto.

4. Em quais situações foi necessário verificar ou corrigir respostas?

A verificação sempre é necessária, principalmente ao lidar com código ou informações que necessitam precisão e certeza. Muitas vezes é preciso fazer pequenas alterações na resposta para adequá-la ao seu caso de uso, principalmente ao utilizar prompts menos descritivos.

5. Você utilizaria essa ferramenta em disciplinas futuras? Por quê?

Sim, inclusive já utilizo a ferramenta nas disciplinas atualmente. Em geral, os resultados foram muito bons e bem explicados. O uso da ferramenta poupa esforços para pesquisar sobre conteúdos complexos e densos e pode gerar materiais rapidamente.